

MetaMorph-AI — Architecture Note

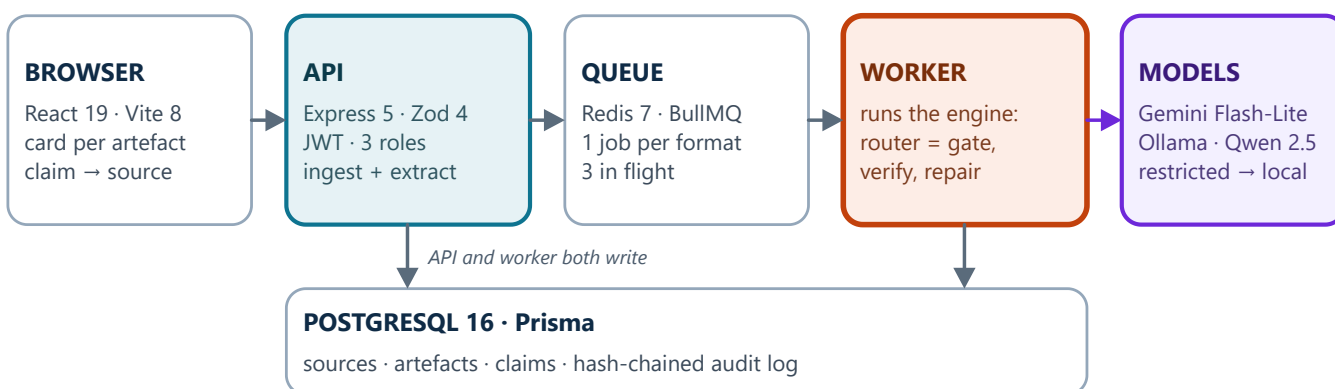
SIH26154 · Gen AI Platform for Automated Content Transformation · NTRO · Blockchain & Cybersecurity · StrawHats · 120014 · 29 Sep 2026

1 Problem and approach

An analyst writes one threat report; the sector CERT, ministry leadership and the public each need it in a different shape. Rewriting it by hand takes most of a working day, and every rewrite can quietly change a fact — “consistent with a possible phishing campaign” becomes “was attacked by a phishing campaign”, or 37 becomes 42.

MetaMorph-AI turns one trusted source into several audience-specific artefacts while keeping every sentence traceable to the passage it came from. It works in four moves: **understand** the source once into a cited fact index; **transform** each format from that index, never from the raw text; **verify** every claim in code; **prove** the result with click-to-source provenance, a tamper-evident log and a human approval.

2 System architecture



- **Browser** (React 19, Vite 8): one independent state machine per artefact card, so one failure never blanks the page. Selecting any claim highlights its source sentence; progress arrives over a WebSocket that resumes from the last event after a drop.
- **API** (Express 5, Zod 4): JWT sessions, three roles, ingestion of text, PDF (`unpdf`, page-aware) and DOCX (`mammoth`), sentence spans via `Intl.Segmenter`. A batch returns 202 in milliseconds; the fact index is extracted inside the ingest request.
- **Queue and worker** (BullMQ 6, Redis 7): one job per artefact, three in flight. Progress goes to Redis Streams — not Pub/Sub — so a reconnecting browser replays what it missed.
- **PostgreSQL 16** (Prisma 6): sources, spans, the fact index, artefacts, claims and the hash-chained audit log.
- **One contract**: the same Zod schemas define every request, event and model output for the browser, the API and the engine — there is no hand-written mirror to drift.

3 The AI engine

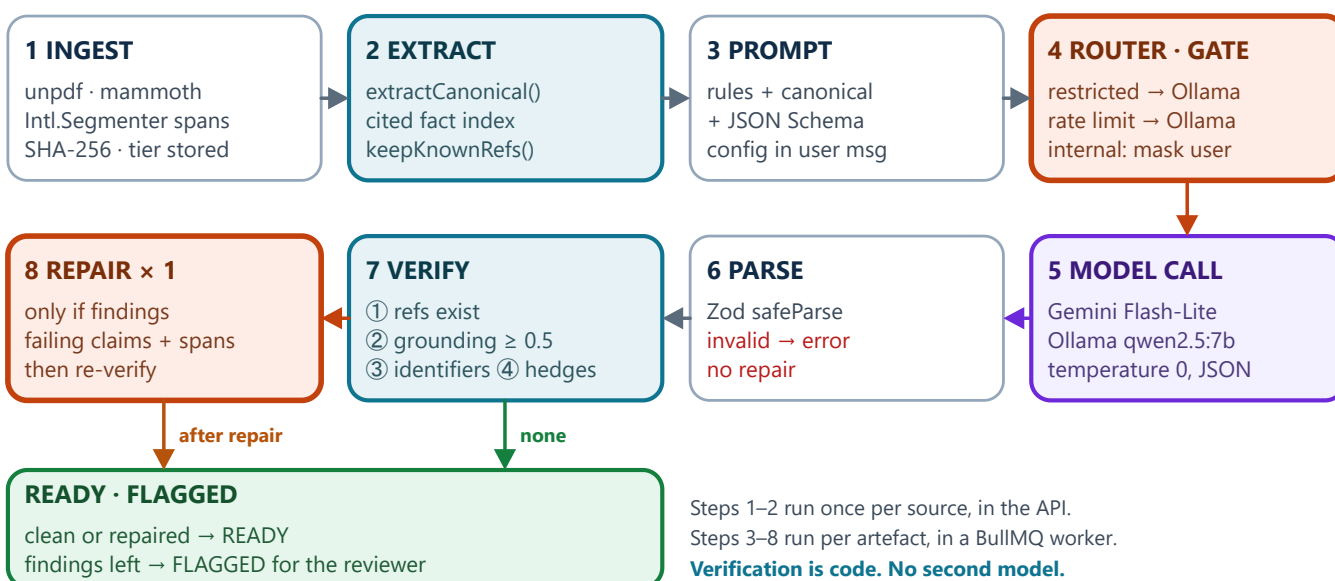


Figure 2 — From source to a ready or flagged artefact. Steps 1–2 run once per source; steps 3–8 run once per artefact.

Extraction reads the source as delimited data in the user role and returns a fact index — severity, entities, events, systems, indicators, key facts (*fact* or *inference*) and recommendations — each item citing span ids. Any id the model invents is dropped. **Generation**

uses one prompt and one schema per format (advisory, executive summary, LinkedIn post); every assertion is a claim node {text, source_refs, status}. Configuration (audience, tone, detail, language) is merged per format, so one artefact can be conversational while another stays formal.

Routing is decided in code before any call. `restricted` goes to the on-device model (Ollama, `qwen2.5:7b`). Otherwise the cloud model (Gemini, `GEMINI_MODEL`, temperature 0) is used, with three tries, falling back to Ollama on a rate limit or repeated failure. Output must satisfy the format's JSON Schema.

Verification is plain code — no second model. Five checks run on every claim: its *source references exist*; its *lexical overlap* with the cited sentence is at least 0.5; every *number, CVE, IP, domain and hash* in it appears in the cited sentence; any *hedge* in the cited sentence (*possibly, potentially, consistent with...*) survives; and the claim is not *cut off mid-sentence*. If anything fails, **exactly one** targeted repair call carries the specific findings and only the cited sentences; the claims are re-checked and a second repair is never made. What still fails is shown on the card as an open flag, and a reviewer approves or rejects. A hard cap of two model calls per artefact is enforced in code and tested.

4 Security and integrity

Property	Mechanism
Sovereignty	The operator classifies each source <i>public, internal or restricted</i> . The check in <code>router.call()</code> runs before an adapter is chosen; restricted content is sent to the on-device model only.
Data, not instructions	Source text is delimited data in the user role; output is confined to a schema; injected text that reaches the output has no valid citation and is shown as <i>unverified</i> .
Access	bcrypt passwords, signed JWTs, operator / reviewer / admin roles enforced on each route. An operator cannot approve their own artefact (HTTP 403).
Tamper evidence	Each audit row stores SHA-256 over its content and its predecessor's hash. A database trigger refuses <code>UPDATE</code> and <code>DELETE</code> ; if a superuser bypasses it, <code>GET /audit/verify</code> names the first broken row. Demonstrated end to end.
Provenance	Spans are split once and never re-derived, so every claim keeps a stable link to its sentence and, for PDFs, its page.

5 Measured results

	Cloud	On-device
Extraction	4 – 31 s	127 – 195 s
Generation per format	24 – 45 s	87 – 131 s
Grounding score	1.00	0.88 – 1.00
Drafts needing a repair	1 of 9	5 of 9

The repair loop earns its place on the weaker model: on-device drafts were repaired five times as often. **“Five artefacts in under 60 seconds” remains a target** — no end-to-end batch has been timed. The grounding score measures citation overlap, not correctness; wrong numbers and dropped hedges are reported in `verification`, not in the score.

Three runs per format, advisory / executive summary / LinkedIn post, measured by the AI owner on 28 September. Automated suite: 61 tests, all passing.

6 Current limits and next steps

- Formats:** advisory, executive summary and LinkedIn post are built end to end. X thread and video package are specified and selectable but return an unverified placeholder.
- Masking:** for *internal* sources, identifiers are masked in the user message — which is the entire source during extraction. Extending this to the generation prompts is the next change.
- Verification scope:** claim text is checked against cited sentences; severity, length and duration constraints are not yet checked in code.
- Audit:** tamper-evidence, not tamper-proofing — deleting the newest rows is undetectable from inside the database. Anchoring the head hash externally (signed digest, C2PA manifest or permissioned ledger) is the roadmap; no blockchain runs in this system.
- Resilience:** no offline cached-pack fallback yet, and review is available through the API with screens still to build.

7 Technology

TypeScript 6 · Node 22 · React 19 · Vite 8 · Tailwind 4 · Express 5 · `ws` · Zod 4 · BullMQ 6 · Redis 7 · PostgreSQL 16 · Prisma 6 · Gemini (Flash-Lite) · Ollama (Qwen 2.5 7B) · `unpdf` · `mammoth` · Turborepo · Vitest · GitHub Actions. Source and setup: github.com/MdFareedKhan01/sih_ps154 — full detail in `docs/system-documentation.md`.